ACCE GPU Group Project: Flood Simulation

1 Problem description

The problem for this assignment is a grid-based, iterative simulation of rainwater flood over a terrain.

The terrain is represented by a two-dimensional *heightmap*, a matrix where each entry corresponds to a discrete location in the landscape, and its value represents the height of the ground in the corresponding location. The water in the simulation originates from a configurable cloud front moving across the terrain, depositing rainfall over time. The assignment includes four predetermined ground scenarios generated with a function: a mountain terrain, a wide valley with hills, and two versions of the same valley with dams of different positions and heights (see Fig. 1 for an example). The simulation computes the flow of water from the highest ground to the lowest ground, leaking out at the scenario boundaries or accumulating in sinks and dams to form pools and lakes.

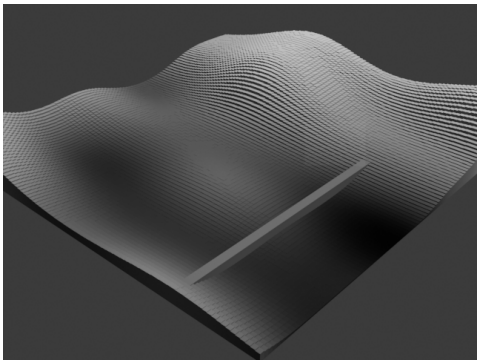

Fig. 1. A 3D render of heightmap 'D'

The simulation proceeds in one-minute steps. At each step, the clouds move and release a predetermined amount of water onto the cells below them. Each cell then transfers part of its water to neighboring cells with lower *total height* (i.e., considering both ground elevation and water levels). More water is transferred into cells with a lower total height than their neighbors. In this way, water flows from higher regions to lower ones, gradually reducing height differences and tending towards equalizing levels. Cells at the edge of the terrain also drain part of their water outside the simulation domain. This amount of water is removed from the simulation.

The simulation continues for a fixed number of time steps, specified by a program argument, or until the amount of water transferred between cells falls below a given threshold, indicating that a sufficient equilibrium has been reached.

1.1 Algorithm details

The core part of the algorithm consists of passes over the grid to compute water addition and water flowing between neighboring cells.

On each time step:

(1) The clouds advance across the scenario, based on their direction and velocity. After updating their positions, clouds deposit water on the underlying terrain.

For each cloud, the rainfall is distributed on the area below the cloud, based on the radius of the cloud and the distance to the center.

(2) The water is redistributed among neighbors. For each matrix position, the amount of water that flows to the four neighboring positions is computed proportionally to the relative differences of their total height (ground + water level).

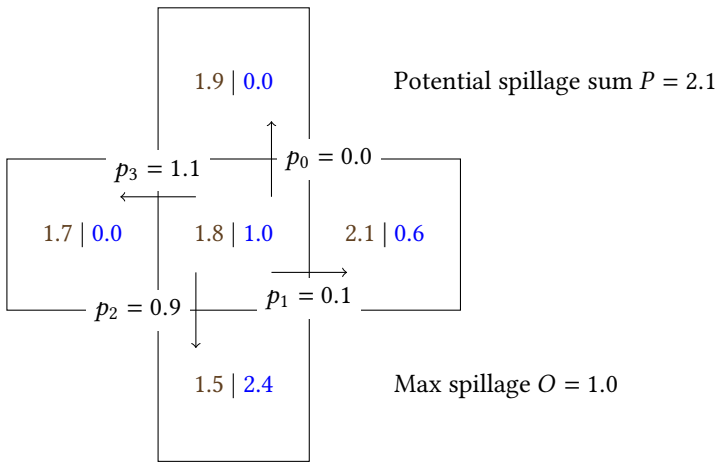

Fig. 2. Step 2: Calculate water spillage. Brown numbers are ground level, blue is the water levels. Arrows indicate the potential spillage.

This is done in two steps. First, we compute for each neighbor the potential spillage p_i between the current cell and the neighbor i (where $i \in 0, 1, 2, 3$ represents respectively the up, right, down, and left neighbor). If the neighbor has an equal or higher total height, the potential spillage is zero ($p_i = 0$).
The total amount of water O that can leave the cell is defined by the maximum spillage between the four neighbors, but limited by the cell water level (W):

$$O = \min(W, \max(p_0, p_1, p_2, p_3))$$

Fig. 2 illustrates an example. In this case, the total spillage of the central cell is given by:

$$O = \min(1.0, \max(0.0, 0.1, 0.9, 1.1)) = \min(1.0, 1.1) = 1.0$$

Then, the actual spillage o_i to each neighbor is computed proportionally, considering the sum of all potential spillages $P = \sum_{i=0}^{3} p_i$:

$$o_i = O \cdot \frac{p_i}{P} \text{ if } P > 0; \text{ 0 otherwise}$$

The actual spillages for the example in Fig.2 are thus:

$$o_0 = 1.0 \cdot \frac{0.0}{2.1} = 0$$
$$o_1 = 1.0 \cdot \frac{1.1}{2.1} \approx 0.52$$
$$o_2 = 1.0 \cdot \frac{0.9}{2.1} \approx 0.43$$
$$o_3 = 1.0 \cdot \frac{0.1}{2.1} \approx 0.05$$

(3) Each matrix cell is updated with the water spillage values from its neighbors, removing the water from its old cell. This flow is divided by `SPILLAGE_FACTOR = 2` so that situations like in Fig.3 won't occur, which would prevent convergence.

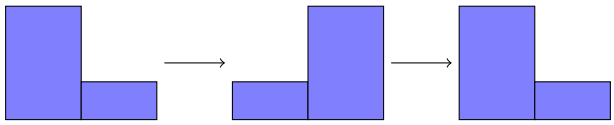

Fig. 3. Water flowing back and forth between adjecent cells without `SPILLAGE_FACTOR`.

The cells are updated sequentially after the flow calculation to ensure all spillage values are calculated before the water levels update, as not doing this would make the results dependent on the update order.

The algorithm repeats steps 1 through 3 across the whole grid until one of the following conditions is satisfied:

- No cell exceeded a minimum flow value.
- A maximum number of iterations is reached.

Out-of-bounds cells are considered to be dry cells with the same ground height as their adjacent in-bound cell. This ensures that water flowing toward the boundaries behaves consistently with the flow rules described above.

The clouds are initialized in the framework code as they are based on either randomness or input parameters, which do not benefit from parallelization. However, you are encouraged to consider optimizations in data storage or memory layout if appropriate.

After the simulation is completed, the program prints the execution time (excluding the initialization phase) and a number of accumulated statistics that help verify the correctness of the results.

We want you to implement a parallel, GPU-based implementation of this algorithm with CUDA. We suggest you not change the algorithm described above in your parallel code. Make sure to use the provided reference implementation to study the algorithm and ensure correctness.

Note: This assignment contains this PDF and a code package. Read carefully the `README` file provided with the code package for further guidance on working with the project, including input arguments, output of the simulation, and how to verify your result. Be aware that parallelizing the algorithm in various ways can introduce floating-point errors, even if the workflow remains mathematically the same. Refer to the README for instructions on how to check your implementation

for correctness.

This assignment is based on a modified version of the Peachy Parallel Assignments [1].

2 Requirements

Each group must submit a CUDA-based parallel implementation of the provided sequential rainwater flood simulation algorithm, a report in **max. 4 pages, excluding references** (see 2.1), and a reproducibility artifact, i.e., all the information necessary to run the code and reproduce the results in the report (see 2.2). The goal is to minimize the computation time as much as possible **while still achieving correct execution as defined in the README file**.

2.1 Report Structure

The report must be limited to four 2-column pages, excluding the bibliography, using the provided template available in the code package and as Overleaf template here. **Make good use of the space you have. Submissions not respecting the constraints will be penalized**.

The report should have the following structure (note: you are allowed to deviate from this as long as the same information is included and it is clear where it is discussed):

(1) *Report title and authors*. Provide the name and contact information (email) for each person in your team.
(2) *Abstract*: a high-level description of your solution, analysis overview, and main results.
(3) *Introduction*: a brief overview of the problem (rain flood simulation), motivation for GPU acceleration, and the structure of the remainder of the article. Use one short paragraph for each.
(4) *Parallelization approach:* In this section, you should explain the design of your parallel solution. For instance (but not limited to):
    - What are the parallelizable sections of the original algorithm, and how did you identify them?
    - How did you distribute the workload (geometry)?
    - Have you applied any particular optimization to improve the performance of your code? Why and how?
    - Any other considerations you might have made, e.g., memory management, workload distribution, etc.
(5) *Implementation details*, if not already included in the previous section.
(6) *Experimental Results:*
    (a) *Experimental setup*: hardware configuration, software details (CUDA version, compiler, etc.), and considered performance metrics.
    (b) *Experiments*: describe the experiments you have conducted to analyze the quality of your solution. This must include at least a study on the scalability of your solution compared to a CPU sequential implementation using various datasets. Ensure that your analysis is statistically significant. Other interesting experiments may include a breakdown of execution time and the impact of different optimizations.
(7) *Discussion and Conclusion*: Here, you can describe the challenges, limitations, and possible optimizations to your solution. We expect you to provide a summary of your findings and (optional but encouraged) lessons learned and a time sheet reporting the time it took to conduct each major part of the assignment.

2.2 Code and reproducibility artifact

The students must provide all the code necessary to run their solution as a compressed ZIP file via Canvas (together with their report). To reproduce the results shown in the report, we ask you to provide:

(1) *Source code:* the complete package, including Makefile.
(2) *README*: containing proper instructions on how to compile and execute your code.
(3) *Scripts*: to reproduce the results in your report. For instance, if you have a scalability plot, your submission must include everything that is needed to *a)* generate the results, and *b)* generate the plot as shown in the report.

3 Running on DAS and Submission

Very Important: you will execute your code on DAS5, which is a shared resource used by multiple users. So:

- **Use it responsibly** to ensure fair access to everyone—allocate nodes only for the time and resources you truly need. Over-allocating prevents others from using the system.
- **Plan ahead:** waiting times might increase as deadlines approach. Avoid last-minute runs to prevent delays and ensure smooth execution of your experiments.

The deadline to submit the code, report, and reproducibility artifact is **Sunday, April the 19th 2026, 23:59 (CET) (strict, no deadline extension)**.

4 Grading

Assignments are graded according to two criteria:

(1) Solution performance and code quality (50% of the assignment grade), including:
    - Correctness of the implementation and applied optimizations.
    - Speedup over CPU implementation for the reference dataset (but not limited to – see README).
(2) Report quality and result reproducibility (50% of the grade), including:
    - solution design, analysis of results, discussion, graphing, writing style and clarity;
    - artifact completeness and ease of use.

References

[1] Almeida, C., Shoop, E., García-Álvarez, D., Gonzalez-Escribano, A., Guerrero-Pantoja, D., Maloney, C., Pantoja, M., Rizzi, S., and Bunde, D. P. Peachy parallel assignments (eduhpc 2025). In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis* (New York, NY, USA, 2025), SC Workshops '25, Association for Computing Machinery, p. 409–415.